Advanced Data Management
Mock Exam v3 (Chapters 1–6): Full Solutions

Every answer in the rubric's shape: the letter or the number first, the formula, the computation with units, the one-sentence decision, the trap

Advanced Data Management

2026-11-01 100 points 2 hours

How to use this document

Mock v3 keeps Mock v2's structure (8 quick checks, 3 situations, 1 real plan, 2 of 3 tickets) on a new database, `citycab`, so that nothing can be answered from memory of v2. Every solution below is written the way the rubric grades, and in the order you should write it on the paper: **Data** (what the question gives), **Formula** (from the sheet), **Numbers** (the substitution, with units), **Result** (one sentence that quotes the number), then the **Trap** (the wrong answer that looks right and why it is wrong). All three tickets of Part D are answered.

Part A: Quick checks

Three points each: one for the letter, two for the justification. The justification is the *Formula* or *Mechanism* line below; the rest is what you would say if asked "why not the others".

Solution: A1 The typo `drvier_id`: who rejects it?

Answer: (b) the binder, the catalog.
Mechanism. The query goes through the five stages in order: parser (grammar), binder (names against the catalog), rewriter, planner (statistics), executor (heap and indexes). `drvier_id` is a perfectly grammatical identifier, so the parser accepts it. The binder looks `drvier_id` up in `pg_attribute` for table `rides`, finds nothing, and raises "column drvier_id does not exist".
Why not the others. (a) the parser never knows which columns exist; (c) the planner is never reached; (d) the executor only runs a finished plan.
Trap: "it is a name error, so it is the parser": the parser checks shape,

the binder checks existence.

Solution: A2 Rows per page of `rides`

Answer: (c) 120.
Data. Column data 40 B, tuple header 24 B, slot 4 B, page header 24 B, page 8,192 B.
Formula. rows per page $= \lfloor (8{,}192 - 24)/(\text{record} + 4) \rfloor$ with record = columns +24, rounded up to a multiple of 8.
Numbers. record $= 40 + 24 = 64$ B (already a multiple of 8); per row on the page $= 64 + 4 = 68$ B; usable $= 8{,}192 - 24 = 8{,}168$ B; $\lfloor 8{,}168/68 \rfloor = \lfloor 120.1 \rfloor = 120$.
Check. $2{,}000{,}000/120 = 16{,}667$ pages, exactly the table's size in the data sheet.
Why not the others. (a) $8{,}168/40 = 204$ forgets header and slot; (b) $8{,}192/64 = 128$ forgets the slot and the page header; (d) is the row count.
Trap: dividing by the column bytes alone; the 24-byte header and the 4-byte slot are paid by every row.

Solution: A3 Page reads for one lookup in `rides_pkey`

Answer: (b) 4.
Data. $N = 2{,}000{,}000$ entries, fanout $F = 200$.
Formula. levels $= \lceil \log_F N \rceil$; lookup = levels +1 heap page.
Numbers. $200^2 = 40{,}000 < 2{,}000{,}000 \leq 200^3 = 8{,}000{,}000$, so $\log_{200} 2{,}000{,}000$ is between 2 and 3 and the ceiling is 3 levels (root, one inner level, leaves). Lookup $= 3 + 1 = 4$ page reads.
Why not the others. (a) forgets the heap page; (c) and (d) use $\log_2$ ($2^{21} \approx 2$M): that is a binary tree, not a B$^+$-tree with 200 children per node.
Trap: the heap page. An index lookup ends in the table unless the index covers every column needed.

Solution: A4 `WHERE lower(email) = 'x@y.ma'`

Answer: (b) a B-tree on `lower(email)`.
Mechanism. An index is sorted (or hashed) on its *key expression*. The predicate compares the value of `lower(email)`, which is not the value of `email`, so an index on `email` cannot be searched: the planner would have to compute `lower` on every row, which is a Seq Scan. An expression index stores `lower(email)` itself, so the predicate becomes an equality on the stored key and the planner does one descent.
Why not the others. (a) and (c) index the wrong key; (d) BRIN stores the min and max of a column per block range and helps only a physically ordered column on a range predicate, never a function on a point lookup.
Trap: "any index on the column helps". The key must match the expression as written.

Solution: A5 Which node cannot emit before consuming everything?

Answer: (c) HashAggregate.
Mechanism. A group's count or sum is final only after the last input row has been seen; until then the node cannot know that a group is complete, so it emits nothing before its input is exhausted. That is the definition of a pipeline breaker.
Why not the others. (a) a Seq Scan with a `Filter` tests each row and passes it on at once; (b) Nested Loop emits a pair as soon as the inner finds a match for the current outer row; (d) Merge Join emits a pair as soon as the two sorted inputs agree. Merge Join needs sorted *inputs*, but the sorting is done by a separate Sort node, which is the breaker, not the join.
Trap: confusing "needs sorted input" with "blocks". The other breakers to remember: Sort, Hash (the build side of a hash join), Materialize.

Solution: A6 External sort of `rides` with $M = 130$

Answer: (c) 66,668.
Data. $B = 16{,}667$ pages, $M = 130$.
Formula. runs $= \lceil B/M \rceil$; merge passes $= \lceil \log_{M-1}(\text{runs}) \rceil$; total I/O $= 2B \times (1 + \text{merge passes})$.
Numbers. runs $= \lceil 16{,}667/130 \rceil = \lceil 128.2 \rceil = 129$. Fan-in $M - 1 = 129$, and $129 \leq 129$, so all runs merge in one pass: merge passes $= 1$. Total passes $= 1 + 1 = 2$. I/O $= 2 \times 16{,}667 \times 2 = 66{,}668$ (33,334 reads and 33,334 writes).
Why not the others. (a) one read of the file; (b) one pass, read and written; (d) assumes two merge passes, which would need more than 129 runs.
Trap: $M = 130$ was chosen so that the fan-in is exactly enough; with $M = 129$ there would be 130 runs, fan-in 128, two merge passes, 100,002 I/Os. Always compare runs with $M - 1$.

Solution: A7 Rows for `status = 'disputed' AND zone_id = 1`

Answer: (a) about 6,000.
Data. `status = 'disputed'` is in the MCV list with frequency 0.01; `zone_id = 1` is in the MCV list with frequency 0.30; $N = 2{,}000{,}000$.
Formula. equality on an MCV value: $s =$ its frequency; AND under independence: $s = s_1 \times s_2$; rows $= s \times N$.
Numbers. $s = 0.01 \times 0.30 = 0.003$; rows $= 0.003 \times 2{,}000{,}000 = 6{,}000$.
Why not the others. (b) $0.01 \times 2\text{M} = 20{,}000$ is `status` alone; (c) $0.30 \times 2\text{M} = 600{,}000$ is `zone_id` alone; (d) is their sum, which is roughly the OR estimate ($s_1 + s_2 - s_1 s_2 = 0.307$, 614,000).
Trap: adding selectivities for AND. And remember this is the planner's *estimate*: if disputes cluster in zone 1 the truth is higher, which is the correlated-columns failure of Part C.

Solution: A8 Clock-sweep meets `usage_count = 3`

Answer: (b) the count becomes 2 and the hand moves on.
Mechanism. PostgreSQL's clock-sweep keeps a `usage_count` from 0 to 5 per frame. A hit increments it (saturating at 5), outside any visit of the hand. When the hand looks for a victim it decrements the count of each unpinned frame it passes and evicts the first frame it finds at 0. A frame at 3 therefore needs three more passes without a hit before it is evicted.
Why not the others. (a) only a count of 0 is evicted; (c) the hand never increments; (d) pinning is a separate counter set by the backend that is using the page, and pinned frames are skipped, not waited for.
Trap: confusing the pin count ("in use right now, never evict") with the usage count ("how recently and often it was used").

Part B: Problem situations

Solution: B1 "An index on a 2 % filter never helps, delete it"

Data. $N = 2{,}000{,}000$ rows, $B = 16{,}667$ pages, $r = 120$ rows per page. `ride_ts` uniform over 365 days, correlation 0.99. `rides_ts_idx`: 3 levels, $L = 300$ entries per leaf. Cost constants: sequential page 1, random page 4, tuple 0.01, index entry 0.005, operator 0.0025.

(a) Selectivity, rows, pages. Formula. range on a uniform column: s = width of the range/width of the column; rows = sN; contiguous pages = $\lceil \text{rows}/r \rceil$.
Numbers. $s = 7/365 = 0.0192$ (1.92 %). Rows $= 0.0192 \times 2{,}000{,}000 = 38{,}356$. Pages, if the rows are stored one after another: $\lceil 38{,}356/120 \rceil = \lceil 319.6 \rceil = 320$.
Result. "The predicate keeps $7/365 = 1.92$ % of the rows, that is 38,356 rows, which fill 320 pages if they are contiguous, which they nearly are since the correlation is 0.99."

(b) Seq Scan versus Index Scan with contiguous rows. Formula. Seq Scan $= B \times 1 + N \times 0.01 + N \times 0.0025$ (pages, tuples, one predicate). Index Scan = descent + leaf pages + heap pages + per-row CPU, with heap pages $\approx sB$ when the correlation is near 1.
Numbers, Seq Scan. $16{,}667 + 20{,}000 + 5{,}000 = 41{,}667$.
Numbers, Index Scan.

Term	Computation	Cost
descent	3 levels × 4 (random)	12
leaf pages	$\lceil 38{,}356/300 \rceil = 128$, chained, read in order ×1	128
heap pages	320 contiguous pages ×1 (sequential price)	320
per-row CPU	$38{,}356 \times (0.005 + 0.01)$	575
total		≈ 1,035

If the grader charges the 320 heap pages at the random price (×4 = 1,280), the total is ≈ 2,000; say which price you used.

Result. "The Seq Scan costs 41,667 units, the Index Scan about 1,035 (at most about 2,000): the index wins by a factor of 40 (20 at worst)."

(c) The same index with correlation 0.003**. Formula.** heap pages touched when matches are scattered $= B\,(1-(1-s)^r)$, each at the random price 4.
Numbers. $(1 - 0.0192)^{120}$: $\ln 0.9808 = -0.01937$; $\times 120 = -2.32$; $e^{-2.32} = 0.098$. So the fraction of pages with at least one match is $1 - 0.098 = 0.902$ and the pages $= 0.902 \times 16{,}667 \approx 15{,}035$, 90 % of the table, about 2.5 matching rows per page. Index Scan $= 12 + 128 + 15{,}035 \times 4 + 575 = 12 + 128 + 60{,}140 + 575 \approx 60{,}855$.
Comparison. $60{,}855 > 41{,}667$: the Seq Scan wins. (A Bitmap Heap Scan would read the same 15,035 pages once, in physical order, about $15{,}035 + 575 + 12 + 128 \approx 15{,}750$: cheaper than both, and that is what PostgreSQL would really pick at this correlation.)
The rule. $s < 1/(4r) = 1/480 = 0.21\,\%$. The filter keeps 1.92 %, nine times above the threshold, so for an *unordered* table the colleague is right.
Result. "With correlation 0.003 the 38,356 rows touch about 15,035 pages, the index scan costs about 60,855, and the Seq Scan at 41,667 wins; the rule $1/(4r) = 0.21\,\%$ predicts exactly that."

Decision. Keep the index. The rule $s < 1/(4r)$ assumes matching rows scattered one per page; with correlation 0.99 the week's rides sit in 320 consecutive pages and the Index Scan costs about 1,000 units against 41,667. The one word is **correlation**: the cost is in pages, not rows. A BRIN index on `ride_ts` would give the same win in a few pages of space, because the table is appended in time order.
Trap: applying $1/(4r)$ without reading `pg_stats.correlation`; the rule is a special case (correlation 0) of the pages formula.

Solution: B2 Driver earnings, sorted by driver

Data. R = `drivers`: $B_R = 160$ pages, $|R| = 20{,}000$. S = `rides`: $B_S = 16{,}667$ pages, $|S| = 2{,}000{,}000$. No index on `rides(driver_id)`. $M = 64$, then 200.

(a) Block nested loop, `drivers` **outer. Formula.** $B_R + \lceil B_R/(M-2)\rceil \times B_S$ ($M - 2$: one buffer for the inner, one for the output).
Numbers. chunks $= \lceil 160/62\rceil = \lceil 2.58\rceil = 3$. Cost $= 160 + 3 \times 16{,}667 = 160 + 50{,}001 = 50{,}161$ page I/Os.
Result. "With $M = 64$ the outer needs 3 chunks, so `rides` is read three times: 50,161 I/Os."

(b) Hash join. Formula. build on the smaller input; if $B_{\text{build}} \le M$: $B_R + B_S$; otherwise Grace: $3(B_R + B_S)$.
Numbers. Build = `drivers`, 160 pages $> M = 64$: does not fit. Grace: partition both inputs on `driver_id` into $M - 1 = 63$ partitions (read and write each input once: $2 \times 16{,}827$), then join partition by partition (read each once: 16,827). Cost $= 3 \times (160 + 16{,}667) = 3 \times 16{,}827 = 50{,}481$.
Result. "The build side does not fit, so the Grace variant applies at 50,481

I/Os, slightly more than the block nested loop, because three chunks of the outer cost the same as Grace's three passes."

(c) Sort-merge join. Formula. per input: runs $= \lceil B/M \rceil$, merge passes $= \lceil \log_{M-1} \text{runs} \rceil$, I/O $= 2B(1 + \text{passes})$; then merge $= B_R + B_S$.

Input	Runs	Merge passes	I/O
`rides`	$\lceil 16{,}667/64 \rceil = 261$	$\lceil \log_{63} 261 \rceil = 2$ $(63 < 261 \leq 3{,}969)$	$2 \times 16{,}667 \times 3 = 100{,}002$
`drivers`	$\lceil 160/64 \rceil = 3$	1 $(3 \leq 63)$	$2 \times 160 \times 2 = 640$
merge			$160 + 16{,}667 = 16{,}827$
total			117,469

Result. "Sorting `rides` alone costs 100,002 I/Os; the sort-merge join totals 117,469, more than twice the other two."

(d) $M = 200$**.**

Join	Computation	I/O
block NL	$\lceil 160/198 \rceil = 1$ chunk: $160 + 1 \times 16{,}667$	16,827
hash	$160 \leq 200$, build fits: $B_R + B_S$	16,827
sort-merge	`rides`: $\lceil 16{,}667/200 \rceil = 84$ runs, 1 pass, $2 \times 16{,}667 \times 2 = 66{,}668$; `drivers`: fits, 320; merge 16,827	83,815

Decision. Run the hash join with `drivers` as the build side. 160 pages = 1.3 MB, so the default `work_mem` of 4 MB already holds it; $M = 64$ was an artificial restriction. The tie with the block nested loop is only in I/O: in memory the nested loop compares every `rides` row with every driver of the chunk, $2{,}000{,}000 \times 20{,}000 = 4 \times 10^{10}$ comparisons, while the hash join does one probe per `rides` row. For the `ORDER BY driver_id`, do not sort the 16,667-page input: aggregate first (HashAggregate over 20,000 drivers, a few hundred kB), then sort the 20,000-row result, a single in-memory quicksort. The plan is Sort ← HashAggregate ← Hash Join.
Trap: sorting the big input to satisfy `ORDER BY`; and using $|R| = 20{,}000$ rows in the nested-loop formula (that is the simple nested loop, $160 + 20{,}000 \times 16{,}667 = 3.3 \times 10^8$, absurd).

Solution: B3 "The morning after the audit"

Data. Pool = 131,072 frames (1 GB). Working set 20,000 pages. $h = 0.995$ by day. $T_{\text{hit}} = 0.1\,\mu$s, $T_{\text{miss}} = 100\,\mu$s. Scan of 400,000 pages every night.

(a) Effective access time. Formula. EAT $= h\,T_{\text{hit}} + (1 - h)\,T_{\text{miss}}$.
Numbers, day. EAT $= 0.995 \times 0.1 + 0.005 \times 100 = 0.0995 + 0.5 = 0.60\,\mu$s. Against RAM: $0.60/0.1 = 6$ times slower. Notice that the 0.5 % of misses (0.5 μs) cost five times more than all the hits together (0.0995 μs).
Numbers, morning at 90 %. EAT $= 0.90 \times 0.1 + 0.10 \times 100 = 0.09 + 10 = 10.09\,\mu$s. Slowdown against the daytime value: $10.09/0.60 = 17$ times; against RAM: 101 times.

Result. "At 99.5 % the EAT is 0.60 μs, six times RAM; at 90 % it is 10.09 μs, 17 times slower than during the day."

(b) Plain LRU during the scan. Mechanism. The scan streams 400,000 use-once pages through the pool. After its first 131,072 pages every frame holds a scan page, because each scan page is more recent than every hot page; the remaining 268,928 scan pages keep evicting scan pages. The last use of every working-set page is older than every scan page, so LRU evicts all 20,000 of them. The scan is three times longer than the pool, three times what is needed.
Numbers. Survivors = 0. Morning reload = 20,000 misses $\times 100\,\mu$s = 2 s of pure I/O, paid by the first transactions of the day as a collapsed hit ratio (the "slow morning" of the ticket).
Result. "Under LRU zero hot pages survive, and the morning pays 20,000 misses, about 2 seconds, before the hit ratio recovers."

(c) PostgreSQL's ring. Formula. a Seq Scan of a table larger than `shared_buffers`/4 uses a 32-frame ring.
Numbers. Threshold = 131,072/4 = 32,768 pages (256 MB). 400,000 > 32,768: ringed. Ring = 32 frames (256 kB); scan page k evicts scan page $k - 32$. Pool left for everything else = 131,072 − 32 = 131,040 ≥ 20,000: the working set survives.
What the ring costs the scan. Nothing: every scan page is a compulsory miss (first touch) and would never be hit again, so caching it in the big pool would not have saved a single read.
Result. "The scan is ringed (400,000 > 32,768), it recycles 32 frames, the working set survives, and the scan loses nothing." So on PostgreSQL the slow morning has another cause: the OS page cache, which the ring does not protect, a `COPY` ring (16 MB) or a checkpoint; measure with `pg_buffercache` right after the job.

(d) `shared_buffers = 48GB`. What the scan gains. Nothing: a use-once read of 3.2 GB has hit ratio ≈ 0 at any pool size, and the ring keeps it out of the pool anyway.
What it costs. The OS cache shrinks to under 16 GB minus `work_mem` per connection, and checkpoints must flush up to 48 GB of dirty pages in bursts.
What to measure first. The working set in `pg_buffercache`: 20,000 pages = 160 MB, already six times smaller than the 1 GB pool; and the hit ratio from `pg_statio_user_tables` by hour, to see *when* it drops. The numbers say the pool is not the problem.
Trap: "more cache is always better"; and reading "99.5 %" as "nearly perfect" without computing the EAT.

Part C: Read a real plan

Solution: Ticket OPS-44, question 1: the tree, blocking or streaming

Tree. Limit ← Sort ← HashAggregate ← Nested Loop with two children: outer = Bitmap Heap Scan on `rides` (whose own child is the Bitmap Index Scan on `rides_zone_idx`), inner = Index Scan on `riders` using `riders_pkey`, run once per outer row (`loops=40000`).

Node	All input first?	Why
Bitmap Index Scan	**yes**	builds the complete bitmap of matching row ids before the heap scan can start
Bitmap Heap Scan	no	walks the bitmap page by page and emits rows as it goes
Index Scan riders	no	one lookup per outer row, emits the match immediately
Nested Loop	no	emits each (ride, rider) pair as soon as the inner returns
HashAggregate	**yes**	a rider's count is final only after the last ride
Sort	**yes**	the largest count is known only after all groups are in
Limit	no	passes the first 20 rows it receives, then stops

Why the LIMIT does not help. Two breakers sit below it: the Sort cannot hand over its first row before the HashAggregate has finished, and the HashAggregate cannot finish before the whole join has run (40,000 inner lookups). "Stop after 20" only saves the sort's output phase, which is why the method is `top-N heapsort` in 27 kB. Read it off the times: the Nested Loop ends at 1,890 ms, the Sort starts at 1,948 ms, the Limit at 1,948 ms.
Trap: calling the Bitmap Heap Scan blocking (it streams) or the Bitmap Index Scan streaming (it does not: the bitmap must be complete so that the heap is read in physical order).

Solution: Ticket OPS-44, question 2: the estimates

`rows=40000` on the Bitmap Index Scan. `zone_id = 7` is in the MCV list with frequency 0.02: $0.02 \times 2{,}000{,}000 = 40{,}000$. Actual 40,000: exact, as MCV estimates are.
`rows=4000` on the Bitmap Heap Scan. The `Filter pickup_city = 'Rabat'` has MCV frequency 0.10, and the planner multiplies under independence: $40{,}000 \times 0.10 = 4{,}000$.
Actual. 40,000, with `Rows Removed by Filter: 0`: every zone-7 ride is in Rabat, because a zone lies inside exactly one city, so the second predicate removes nothing.
q-error. $\max(\text{est}/\text{actual}, \text{actual}/\text{est}) = 40{,}000/4{,}000 = 10$, an *under*-estimate, the dangerous direction: it makes the planner pick per-row plans (nested loop, index lookups) for an input that is ten times bigger than it

thinks.

Failure. Correlated columns (failure 2 of the three): `zone_id` functionally determines `pickup_city`, and the independence rule $s_1 s_2$ is wrong by exactly the factor $1/s_2 = 10$.

Does `ANALYZE rides` fix it? No. Both single-column frequencies (0.02 and 0.10) are already fresh and correct; the error is the multiplication, which no amount of re-sampling changes.

The fix.

```
CREATE STATISTICS s_geo (dependencies) ON zone_id, pickup_city
    FROM rides;
ANALYZE rides;
```

The planner then knows that `pickup_city` is implied by `zone_id` (dependency degree ≈ 1) and estimates $0.02 \times 2{,}000{,}000 \times 1 \approx 40{,}000$ for the pair.

Result. "The Bitmap Index Scan keeps $0.02 \times 2{,}000{,}000 = 40{,}000$ rows, then the Filter multiplies by 0.10 to 4,000; the truth is 40,000, q-error 10, correlated columns, fixed by extended statistics, not by ANALYZE."

Solution: Ticket OPS-44, question 3: the Buffers numbers and the two seconds

`Heap Blocks: exact=15190`.

Formula. pages with at least one match $= B\,(1 - (1 - s)^r)$ when the correlation is near 0 (here 0.003).

Numbers. $s = 0.02$, $r = 120$: 0.98^{120}: $\ln 0.98 = -0.0202$; $\times 120 = -2.42$; $e^{-2.42} = 0.089$. Pages $= 16{,}667 \times (1 - 0.089) = 16{,}667 \times 0.911 = 15{,}190$, 91 % of `rides`, about $40{,}000/15{,}190 = 2.6$ matching rows per page. The bitmap scan reads each of them exactly once, in physical order; `Buffers: shared hit=15193` (15,190 heap pages +3 index pages) says they were all in the pool: `rides` is warm.

`Buffers: shared hit=121400 read=38600` on the inner Index Scan.

Formula. pages per lookup = levels +1; total = loops × pages per lookup.

Numbers. `riders_pkey` has 3 levels, so each lookup touches 3 index pages and 1 heap page = 4; $40{,}000 \times 4 = 160{,}000 = 121{,}400 + 38{,}600$. The root, the inner level and most leaves are hot and hit. The heap of `riders` is 20,000 pages (160 MB); the query touches 31,207 distinct riders, spread over the heap at random, and 38,600 of those heap fetches miss the pool (some pages are fetched more than once, hence 38,600 and not 31,207).

Where the two seconds go. 38,600 misses × $\approx 50\,\mu$s = 1.93 s, which matches `actual time` 1,890 ms on the Nested Loop against 310 ms for the whole bitmap scan. The cost is the random reads of `riders` heap pages, one per outer row.

Is the hit ratio useful? $h = 136{,}593/(136{,}593 + 38{,}600) = 136{,}593/175{,}193 = 78\,\%$. Not useful: it averages 15,193 cheap sequen-

tial hits on `rides` with 38,600 expensive random misses on `riders`; the cost is in the `read` count on one node, and a ratio hides it. Read `Buffers` per node, not per query.
Trap: "`read` means disk". `read` means "not in shared buffers"; some of the 38,600 came from the OS cache, which is why the average is 50 and not 100 μs.

Solution: Ticket OPS-44, question 4: the plan the planner should have chosen

Why the nested loop was chosen. The planner believed 4,000 outer rows: $4{,}000 \times 4.07 = 16{,}280$ units of inner lookups, cheaper than scanning `riders`. With the true 40,000 rows the lookups cost $40{,}000 \times 4.07 = 162{,}800$ units; add the bitmap scan (17,833): the plan as run is worth about 180,600 units, and it took 1.95 s.
The hash join.
Build side. The smaller input after filtering: `rides` filtered to 40,000 rows $\times$ about 70 bytes = 2.8 MB $<$ 4 MB `work_mem`: one batch, `Batches: 1`.
Probe side. A Seq Scan of `riders`: 20,000 pages $\times 1 + 2{,}000{,}000 \times 0.01 =$ 40,000 units.
Total. bitmap scan 17,833 + seq scan 40,000 + hash and probe CPU $0.0025 \times (40{,}000 + 2{,}000{,}000) \approx 5{,}100$: about 63,000 units, 2.9 times less than 180,600.
Time. Better than the ratio says: the probe reads 20,000 *sequential* pages (160 MB at about 500 MB/s $\approx$ 0.3 s) instead of 38,600 random pages; with the 0.3 s bitmap scan, about 0.6 s instead of 1.95 s.
Plan shape. `Hash Join` with the Hash node over the filtered Bitmap Heap Scan of `rides` (build), probed by a `Seq Scan on riders`: the join order flips, `rides` becomes the inner side.
Result. "With 40,000 outer rows the planner would choose a hash join, build on the filtered `rides` (2.8 MB, one batch), probe with a Seq Scan of `riders`: about 63,000 units and 0.6 s against 180,600 units and 1.95 s."
Trap: building the hash on `riders` (20,000 pages, 160 MB, 40 batches at 4 MB).

Solution: Ticket OPS-44, question 5: the two proposals

(i) `CREATE INDEX ON rides (zone_id, pickup_city).`

Plan lines	the `Filter` line disappears: `pickup_city = 'Rabat'` moves into the `Index Cond` of the Bitmap Index Scan
Heap Blocks	unchanged, 15,190: the same 40,000 rows live on the same pages; the index only decides which row ids go in the bitmap
Estimate	unchanged, 4,000: a two-column index condition is costed with the same independence rule
Time	unchanged: the inner loop still runs 40,000 times (1.9 s); the only effect is a slower insert on `rides`

(ii) `CLUSTER rides USING rides_zone_idx.`

What it does	rewrites the table in `zone_id` order, one-off, 130 MB, under an exclusive lock; correlation of `zone_id` becomes ≈ 1 (and that of `ride_ts` drops)
Heap Blocks	the 40,000 rows now sit in $\lceil 40{,}000/120 \rceil = 334$ consecutive pages: 15,190 $\rightarrow$ about 334
Plan lines	the planner may replace the bitmap scan by a plain Index Scan on `rides_zone_idx` (correlation high), the `Filter` line stays
Estimate	unchanged, 4,000
Time	the scan drops from 310 ms to a few ms; the 40,000 random `riders` lookups remain: about 1.65 s instead of 1.95 s; new rides append out of order and the benefit decays

Decision. First the extended statistics of question 2: two commands, no rewrite, no lock, and they fix the join, which is where the time is (1.95 s $\rightarrow$ about 0.6 s). Then, if zone dashboards are the main workload, consider CLUSTER or a partial index per hot zone for the remaining 0.3 s.
Trap: fixing the scan when the join is the cost. Always locate the time first (question 3), then choose the fix that moves *that* number.

Part D: Discussion

A full-mark answer has four parts: claim, mechanism, number, consequence. Each solution below is written in that order; the bold labels are the structure the grader looks for.

Solution: Ticket D1 "Move rides to a column store"

Claim. Column storage helps the analytics scan by roughly a factor of eight and hurts every OLTP operation by a factor of 25; the answer is both, not one.
Mechanism and numbers, analytics. The query touches 3 of the 25 columns. In a row store it must read every page, 16,667 (130 MB), because the three columns live inside every row. In a column store each column is a separate file, so it reads $3/25 = 12\,\%$ of the bytes, about 2,000 page-equivalents; and column files of similar values compress well (dates and zone ids by run-length or delta), often another factor of 3 to 5: 16,667 pages become a few hundred.
Mechanism and numbers, the app. An insert of one ride must append to 25 column files instead of writing one slotted-page tuple: 25 writes instead of 1 (and 25 `fsyncs` under WAL). At 2,000 inserts per second that is 50,000 column writes per second against 2,000. A point read of a ride by `id` reassembles the row from 25 places, 25 random reads instead of 4 (3 index levels and 1 heap page).
Consequence and recommendation. Keep `rides` as a row store for the app (OLTP: short transactions, whole rows by key, B-trees). Give analytics a column copy: export nightly with `COPY (SELECT zone_id, ride_ts, fare FROM rides WHERE ride_ts >= current_date - 1) TO '/data/rides.parquet' (FORMAT parquet)` and query it with

DuckDB (`read_parquet`), which reads only the referenced column chunks and row groups. Inside PostgreSQL, two cheaper steps first: a BRIN index on `ride_ts` (the table is append-ordered) so that per-day reports read only that day's block ranges, and a covering index `ON rides (ride_ts) INCLUDE (zone_id, fare)` for an index-only scan (`Heap Fetches: 0`).
Trap: "column stores are simply better": they are worse for single-row reads and writes by the number of columns.

Solution: Ticket D2 "Replace clock-sweep with true LRU"

Claim. LRU is not optimal, exact LRU is too expensive to be worth its small gain, and the real enemy of every recency policy is the sequential scan.
Mechanism, optimality. The optimal policy is Belady's OPT: evict the page whose next use is furthest in the future. It needs the future, so it is only a yardstick. LRU approximates it by assuming the past predicts the future (recently used $\Rightarrow$ soon used again). The stack property (more frames never cause more misses, unlike FIFO's Belady anomaly) is a nice guarantee, not optimality.
Number, the cost of exact LRU. Exact LRU keeps a doubly linked list ordered by last use; *every hit* unlinks and relinks the frame, under a lock shared by all backends. At a 99.5 % hit ratio, hits are 99.5 % of all page accesses, millions per second on a busy server: the list becomes a global serialization point and the latency of a hit (0.1 μs of memory) is dominated by the lock, not by the memory access.
Mechanism, the approximations. Clock keeps one reference bit per frame and a hand: a hit sets the bit without touching any shared structure; the hand clears bits and evicts the first frame with bit 0, "not used since the hand last passed", a coarse LRU. PostgreSQL's clock-sweep uses a `usage_count` saturating at 5 instead of one bit: a page needs several hand passes without a hit to be evicted, which adds *frequency* to recency (a page hit five times survives a page hit once), still without a lock on a hit.
The failing workload. A sequential scan larger than the pool: every page is used once, so LRU, clock, FIFO, and even OPT given that future, fill the pool with pages that will never be reused and evict the hot working set (sequential flooding). PostgreSQL does not fix this with the policy but with intent: a scan of a table larger than `shared_buffers`/4 runs through a 32-frame ring, `COPY` and `VACUUM` through rings of their own, so the scan recycles its own frames and the hot set survives.
Consequence. Replacing clock-sweep with a list would make every hit slower for a marginal gain in hit ratio and would not touch the one case that actually hurts.
Trap: "LRU is optimal". OPT is; LRU is the best guess without the future.

Solution: Ticket D3 "Set join_collapse_limit = 1 everywhere"

Claim. Freezing the written join order server-wide trades 30 seconds of planning once for bad plans on every query; fix the one query instead.
Numbers. Nine tables have $9! = 362{,}880$ left-deep join orders (more with bushy trees), each combinable with three join algorithms and several access paths. System-R dynamic programming does not enumerate them: it keeps the cheapest plan for every non-empty subset of tables, $2^9 - 1 = 511$ subsets, built level by level (pairs, then triples, ...), plus one extra plan per *interesting order* (a sort order a later merge join or the `ORDER BY` can reuse). That is why 30 seconds is unusual even for nine tables: the cost is usually in interesting orders and in costing many index paths. Beyond `join_collapse_limit` (8 by default) PostgreSQL already stops reordering explicit `JOIN`s, and beyond `geqo_threshold` (12) it switches to genetic search.
What the setting does. `join_collapse_limit = 1` tells the planner to join exactly in the order the `FROM` clause lists the tables; it still chooses algorithms and access paths, but not the order.
Why the written order is rarely the best. The best order joins the most selective inputs first so that intermediate results stay small: selection pushdown shrinks each table to its filtered rows before the join, and the planner wants the smallest filtered input as the build or outer side of the first join. A developer writes tables in the order of the business question, not of their filtered sizes; if the 2,000,000-row table with no filter comes first, the first join produces millions of rows that every later join must process, and a join that should produce 40,000 rows produces them after 10^9 intermediate ones. A frozen order is also wrong next month, when the data distribution changes.
Instead. (1) Find out why planning is slow: `EXPLAIN` the query, count the indexes and the interesting orders. (2) If the query is a long report, 30 s of planning against minutes of execution is acceptable, and a prepared statement or `plan_cache_mode = force_generic_plan` plans it once. (3) If the written order is known to be good, `SET LOCAL join_collapse_limit = 1` inside that one transaction, or restructure that query with a materialised intermediate table. (4) Leave the default for the rest of the server.
Trap: 9! as "the number of plans the planner evaluates": it evaluates 511 subsets.

Summary: Where each question lives in the textbooks

- A1: Chapter 1 (life of a query, the binder). A2: Chapter 2 (slotted page, rows per page). A3, A4: Chapter 3 (B^+-tree height, expression indexes). A5, A6: Chapter 4 (pipeline breakers, external sort). A7: Chapter 5 (MCV, independence). A8: Chapter 6 (clock-sweep).
- B1: Chapter 3 §4–5 and Chapter 5 §5 (index versus scan, correlation). B2: Chapter 4 §4–5 (join costs, memory). B3: Chapter 6 §1, §5 (EAT, flooding, rings, sizing).
- C: Chapter 4 §7 (reading a plan), Chapter 5 §4, §7 (independence, the three failures), Chapter 3 §4 (pages touched, CLUSTER), Chapter 6 §7 (the Buffers line).
- D1: Chapters 1–2 (OLTP/OLAP, row/column). D2: Chapter 6 §4–5 (policies, rings). D3: Chapter 5 §2, §6 (pushdown, System-R DP).